

QualiJournal Admin – 재현 가능한 배포 & QA 자동 점검 런북

이 문서는 **QualiJournal Admin** 패널에서 QA 자동 점검 버튼이 작동하지 않거나 컨테이너가 정상적으로 배포되지 않는 문제를 단계별로 해결하고, 최종적으로 **성공 상태를 재현**하는 과정을 정리합니다. 아래 절차는 서울에 위치한 배포 환경(Asia Northeast 3)과 도쿄에 위치한 도메인 매핑 환경(Asia Northeast 1)을 모두 고려합니다.

1. 준비 – 환경 변수 및 토큰

1. **Google Cloud CLI**에 로그인한 상태인지 확인한다.
2. PowerShell 또는 터미널에서 다음 환경 변수를 정의한다. 서비스명이나 리전은 실제 환경에 맞게 수정해도 된다:

```
# 프로젝트 및 서비스 정의
$PROJECT="quali-journal-prod"
$REG_SEO="asia-northeast3" # 서울 리전 (소스 배포)
$REG_TYO="asia-northeast1" # 도쿄 리전 (도메인 매핑)
$SRC="quali-journal-admin" # 메인 소스 서비스
$LIVE="quali-admin-domap" # 도메인 매핑 서비스

# ADMIN_TOKEN을 비밀에서 읽어 변수에 저장
$TOKEN = (gcloud secrets versions access latest --secret=ADMIN_TOKEN --
project $PROJECT | Out-String).Trim()
```

3. 로컬 파일 구조는 `C:\Users\user\Desktop\퀄리저널\admin` 을 **배포 소스 루트**로 한다.

2. 구버전 UI 확인

1. 브라우저에서 `https://admin.standardai.co.kr/?v=$(Get-Random)` 을 열어 Admin UI를 확인한다. URL 끝의 `?v=random` 파라미터는 캐시를 무시하기 위함이다.
2. 저장한 HTML을 검사하여 다음 두 지문이 모두 있는지 확인한다:
3. `safewire("btn-qa", "runQa")`
4. `id="btn-qa"`
5. 둘 중 하나라도 발견되지 않으면 **컨테이너 이미지에 HTML 파일이 누락**된 것이다. 아래 3 단계로 이동한다.

3. HTML 및 파이프라인 스크립트 포함

컨테이너 이미지에 HTML 파일과 백엔드 스크립트가 포함되어야 최신 UI와 QA 자동 점검이 작동한다.

3-1. `.gcloudignore` 와 `.dockerignore` 수정

프로젝트 루트(`admin` 디렉터리)에 존재하는 `.gcloudignore` 및 `.dockerignore` 파일에 아래 규칙이 없다면 추가한다:

```
!index.html
!index_lite_black.html
!**/*.html
!*.py
!**/*.py
```

이 규칙은 HTML과 파이썬 스크립트가 빌드 컨텍스트와 컨테이너 이미지에 포함되도록 보장한다.

3-2. 파이프라인 스크립트 복사

오케스트레이터와 관련 폴더가 `admin` 빌드 컨텍스트에 없으면 복사한다.

```
cd "C:\Users\user\Desktop\컬리저널"
Copy-Item .\orchestrator.py .\admin\ -Force
if (Test-Path .\tools) { Copy-Item .\tools .\admin\tools -Recurse -Force }
if (Test-Path .\feeds) { Copy-Item .\feeds .\admin\feeds -Recurse -Force }
if (Test-Path .\data ) { Copy-Item .\data .\admin\data -Recurse -Force }
```

4. IAM 권한 구성

Cloud Run에서 컨테이너 이미지를 가져오지 못하는 `Revision is not ready... Container import failed` 오류는 대부분 IAM 권한 때문이다. 아래와 같이 서비스 계정별로 권한을 부여한다.

1. 런타임 서비스 계정 확인:

```
$SA_SRC = gcloud run services describe $SRC --region $REG_SEO --project $PROJECT --format="value(spec.template.spec.serviceAccountName)"
$SA_LIVE = gcloud run services describe $LIVE --region $REG_TYO --project $PROJECT --format="value(spec.template.spec.serviceAccountName)"
```

2. Artifact Registry Reader 및 Secret Manager Secret Accessor 역할을 런타임 서비스 계정에 부여한다.

```
gcloud projects add-iam-policy-binding $PROJECT --member="serviceAccount:$SA_SRC" --role="roles/artifactregistry.reader"
gcloud projects add-iam-policy-binding $PROJECT --member="serviceAccount:$SA_LIVE" --role="roles/artifactregistry.reader"
gcloud projects add-iam-policy-binding $PROJECT --member="serviceAccount:$SA_SRC" --role="roles/secretmanager.secretAccessor"
gcloud projects add-iam-policy-binding $PROJECT --member="serviceAccount:$SA_LIVE" --role="roles/secretmanager.secretAccessor"
```

3. Cloud Build 서비스 계정도 Artifact Registry Writer 권한을 부여한다:

```
$PN=(gcloud projects describe $PROJECT --format="value(projectNumber)")
$CB_SA="$PN@cloudbuild.gserviceaccount.com"
```

```
gcloud projects add-iam-policy-binding $PROJECT --member="serviceAccount:
$CB_SA" --role="roles/artifactregistry.writer"
```

이 단계까지 완료하면 Cloud Run과 Cloud Build가 Artifact Registry에 저장된 이미지를 읽고 쓸 수 있다.

5. 컨테이너 이미지 빌드 및 업로드 (Plan B)

1. **Artifact Registry 리포지토리 생성** – 리포지토리 이름 `quali` 가 없다면 생성한다:

```
gcloud artifacts repositories create quali --repository-format=docker --
location=$REG_SEO
```

2. **이미지 빌드 및 푸시** – admin 디렉터리에서 소스 코드를 빌드하여 이미지로 업로드한다:

```
cd "C:\Users\user\Desktop\퀄리저널\admin"
gcloud builds submit --tag asia-northeast3-docker.pkg.dev/$PROJECT/quali/
qualijournal-admin:stable
```

빌드 과정에서 **index.html** 과 **Python 스크립트**가 이미지에 포함되어야 한다. `.dockerignore` 규칙을 체크했는지 다시 확인한다.

1. **환경 변수로 이미지 경로 저장:**

```
$IMG="asia-northeast3-docker.pkg.dev/$PROJECT/quali/qualijournal-admin:stable"
```

6. 이미지로 배포 및 트래픽 전환

1. **메인 서비스(SRC) 배포** – 새로 빌드한 이미지를 서울 리전의 소스 서비스에 배포한다.

```
gcloud run deploy $SRC --image $IMG \
--region $REG_SEO --project $PROJECT \
--set-secrets ADMIN_TOKEN=ADMIN_TOKEN:latest --allow-unauthenticated \
--set-env-vars "PYTHONUTF8=1,PYTHONIOENCODING=utf-8"
```

2. **도메인 서비스(LIVE) 배포** – 같은 이미지를 도쿄 리전의 도메인 매핑 서비스에 배포한다.

```
gcloud run deploy $LIVE --image $IMG \
--region $REG_TYO --project $PROJECT \
--set-secrets ADMIN_TOKEN=ADMIN_TOKEN:latest --allow-unauthenticated \
--set-env-vars "PYTHONUTF8=1,PYTHONIOENCODING=utf-8"
```

3. **각 서비스의 트래픽을 최신 리버전에 100 % 전환:**

```
gcloud run services update-traffic $SRC --to-latest --region $REG_SEO --
project $PROJECT
gcloud run services update-traffic $LIVE --to-latest --region $REG_TYO --
project $PROJECT
```

7. 헬스 체크 및 API 상태 확인

새 리비전이 활성화되면 아래 커맨드로 서버 상태를 검증한다.

```
# 라이브 서비스의 URL 가져오기
$URL = gcloud run services describe $LIVE --region $REG_TYO --project $PROJECT --
format="value(status.url)"

# /health 엔드포인트는 200을 반환해야 한다
curl.exe -s -D - "$URL/health" | findstr /R "^HTTP/* 200"

# /api/status 엔드포인트도 200을 반환해야 한다 (X-Admin-Token 헤더 포함)
curl.exe -s -D - -H "X-Admin-Token: $TOKEN" "$URL/api/status" | findstr /R "^HTTP/* 200"

# /api/report 는 POST 요청이며 Content-Length: 0 헤더를 추가해야 한다 (411 오류 방지)
curl.exe -s -D - -H "X-Admin-Token: $TOKEN" -H "Content-Length: 0" -X POST "$URL/api/report"
| findstr /R "^HTTP/* 200"
```

모든 엔드포인트가 200 OK 를 반환하면 서버와 토큰 구성이 정상이다.

8. UI 검증 및 QA 자동 점검

1. 브라우저에서 `https://admin.standardai.co.kr/?v=<random>` 로 캐시를 우회해 접속한다.
2. 페이지 상단에 **ADMIN_TOKEN** 을 입력하고 [입력] 버튼을 눌러 초록색 “인증됨” 배지가 나타나는지 확인한다.
3. KPI 새로고침, Ready 불러오기, QA 자동 점검 버튼이 모두 동작하는지 확인한다.
4. 우측 [QA 자동 점검] 버튼을 1회 클릭하면 다음 순서로 모달 로그가 출력되어야 한다:
5. `/api/flow/keyword` : 수집 → 승인Top20 → 발행 순서 실행
6. `/api/items?state=ready` : Ready 목록 조회
7. `/api/items/{id}/publish` : 첫 Ready 아이템 발행
8. `/api/export/md?preview=1` 와 `/api/export/csv` : Markdown/CSV 내보내기 검증
9. QA 자동 점검이 성공하면 모달에 “QA 완료” 메시지가 표시된다.

9. 문제 해결 팁

- **캐시 우회**: 새 리비전을 테스트할 때 항상 URL 끝에 `?v=<random>` 을 붙여 브라우저 캐시를 무시한다.
- **컨테이너 가져오기 실패**: 여전히 `Container import failed` 오류가 발생한다면 서비스 계정 권한 (Artifact Registry Reader, Secret Manager Access)과 이미지 빌드 규칙을 다시 확인한다.
- **HTML 또는 스크립트 누락**: 컨테이너 로그에 `index.html` 또는 `orchestrator.py not found` 오류가 보이면 `.gcloudignore` / `.dockerignore` 설정과 파이프라인 스크립트 복사를 다시 점검한다.

결론

위 절차는 환경 변수 설정 → HTML 및 스크립트 포함 → IAM 권한 부여 → 이미지 빌드/배포 → 트래픽 최신화 → 헬스 체크 → UI 및 QA 검증의 순서로 진행된다. 이 런북을 따라 실행하면 **QualiJournal Admin** 서비스가 최신 UI로 배포되고, QA 자동 점검 버튼을 통한 수집→승인→발행→Export 파이프라인이 완전히 정상적으로 작동한다. 또한, IAM 권한과 캐시, 빌드 컨텍스트 문제를 동시에 해결하여 같은 문제가 재발하지 않도록 한다.
